Day 7 Student Notes: Claude for Linux Administration, Part 2

> **Today in one line:** Claude drafts safer service files, schedules and scripts. **You review, test and decide.**

systemd: keeps your app running

```
[Unit]
Description=ShopEasy API

[Service]
User=shop
ExecStart=/usr/bin/python3 /opt/shop/api.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

[Unit] = who am I. [Service] = how to run me. [Install] = when to start me.

Command	What it does
systemctl status x	Is it running? (only looks)
systemctl restart x	Stop and start
systemctl enable x	Start at boot
journalctl -u x	Its log (only looks)

Five lines that make a service safer

Line	Why
User=shop	Not root. An attacker gets less power
Restart=on-failure	Comes back after a crash
NoNewPrivileges=true	Cannot gain extra rights
ProtectSystem=strict	Most of the disk is read-only (add ReadWritePaths for folders it must write)
PrivateTmp=true	Its own private /tmp

Secrets do not belong in the file. Test new service files on a practice machine first.

cron: runs a job on a schedule

```
minute hour day month weekday  command
   0     3    *    *     *      /opt/backup.sh
```

Schedule	Meaning
0 3 * * *	Every day at 03:00
*/15 * * * *	Every 15 minutes
0 9 * * 1	Mondays at 09:00
* * * * *	Every minute (usually a mistake)

Common mistakes: every minute by accident, no full path to the script, no log of what happened, deleting with rm -rf. **crontab -l** shows your schedule.

Firewall and ports

Port	Used for	Open to the internet?
22	SSH (admin login)	Only if needed, locked down
80 / 443	Website	Yes
3306	MySQL database	No
6379	Redis	No

sudo ufw status shows the firewall. **ss -tlnp** shows which ports are listening. Rule: open only what the world needs.

Hardening checklist

Check	Good setting
Root SSH login	PermitRootLogin no
SSH passwords	PasswordAuthentication no (use keys)
Firewall	On, only needed ports open
Services	Run as their own user, not root
Updates	Security updates applied

Checklists such as the CIS Benchmarks have hundreds of checks like these. Claude can compare a file to a checklist, but it is a helper, not a certified audit.

> **Safety rules on a real server:** keep a second SSH session open before changing the firewall or SSH. Test on a practice machine first. No real passwords in files or prompts. Humans apply the changes.

Hands-on (prompts also in Day07_Prompts_To_Paste.txt)

Setup: unzip **server-hardening.zip**, open the folder in VS Code, open the Claude panel. Use **Plan** mode for reading and **Manual** when Claude creates files. Nothing runs on a server today.

1. Explain the files

```
Role: You are a patient Linux teacher.
Context: shop-api.service is a systemd service file. crontab.txt is a cron
schedule. Both are made up.
Task: Explain each file line by line in simple words. Then tell me 3 things that
look weak or risky.
Constraints: Read only. Do not edit anything. No jargon without a one-line
explanation.
Format: A short list per file. Then the 3 risks as bullets.
```

2. Generate hardened files

```
Role: You are a careful Linux engineer.
Context: shop-api.service and backup.sh are weak, made-up files. The API runs
/opt/shop/api.py and needs only /var/lib/shop and /var/log/shop.
Task: Create shop-api.hardened.service with safer settings. Create backup_safe.sh
and one cron line for it, at 03:00 every day.
Constraints: Do not change the original files. Do not run anything. No real
passwords: use a placeholder. Backup file names include the date. Never delete
other backups. Add a comment on each important line.
Format: Two new files plus the cron line in a file called crontab.new. Then 5
bullets: what you changed and why.
```

Check each diff. Then in the terminal: **bash -n backup_safe.sh** (checks typing, does not run it).

2b. Review Claude's own work

```
Role: You are a strict reviewer who checks other people's work.
Context: Review the files you just created: shop-api.hardened.service,
backup_safe.sh and crontab.new.
Task: Find any mistakes, risks or missing safety checks. Rate each finding High,
Medium or Low.
Constraints: Read only. Do not fix anything yet. Be specific: file, line, problem.
Format: A table: file, line, problem, rating. Then the 3 fixes you would do first.
```

3. Review firewall, ports and SSH

```
Role: You are a cautious security engineer.
Context: ufw-status.txt is the firewall, ports.txt shows which ports listen, and
sshd_config.txt is the SSH setting. All made up. Port 3306 is the database, 6379 is
Redis, 8080 is the app behind nginx.
Task: Compare them with a basic hardening checklist. Say which ports should be open
to the internet, which should not, and which SSH settings are weak.
Constraints: Read only. Do not run anything. Never suggest a change that could lock
me out of SSH without warning me first.
Format: A table: item, current setting, risk, safer setting. Then the order I
should change things in.
```

4. Runbook to script

```
Role: You are a careful DevOps engineer.
Context: runbook.md lists the manual steps to restart shop-api. This is a practice
server.
Task: Turn it into a bash script, restart_shop_api.sh. It stops and prints a clear
message if any check fails.
Constraints: It must have a --dry-run option that only prints what it would do. Use
set -e and quote all variables. Do not run it.
Format: One script file with a comment above each step. Then 3 bullets on what
could still go wrong.
```

Check for **--dry-run**, **set -e** and quoted variables. Do not run it.

Key words

Word	Meaning
Service	A program that runs in the background
systemd	Starts and manages services on Linux
cron	Runs jobs on a schedule
Port	A numbered door a program listens on
Firewall	Decides which doors are open
Hardening	Removing ways an attacker could get in
Dry run	Shows what would happen without doing it

Check yourself

1. Why should a service not run as root?
2. What does 0 3 * * * mean?
3. Which port is MySQL on, and should it face the internet?
4. Name two SSH settings that make a server safer.
5. Why ask Claude to review its own draft?

Answers

1. If the app is hacked, the attacker gets full control.
2. Every day at 03:00.
3. 3306. No.
4. PermitRootLogin no, PasswordAuthentication no (use keys).
5. A second look catches mistakes. You still decide.

Take-home task (20 to 30 minutes)

1. Ask Claude to write a systemd timer instead of the cron line, then review it with a second prompt.
2. Bring one change from today you would not apply on a live server without testing, and why.

> **Next class, Day 8:** Claude for Git: everyday workflow.